# Data Structures and Algorithms

Gabriel Kinshuk

2026

# Contents

# 1 Foundations

Algorithm design draws on a small set of general strategies. Three of them account for most of the problems here: greedy choice, divide and conquer, and dynamic programming.

Most of these problems follow the same pattern. We begin with a correct brute-force method, identify why it is slow, and replace it with a method whose running time we can bound. The three paradigms are the recurring shapes those faster methods take.

### Measuring running time

The wall-clock running time of a program depends on the machine, the language, and the compiler. Counting seconds is not portable across any of these. We instead count elementary operations, meaning arithmetic, comparisons, and array accesses, as a function of the input size $n$. By $T(n)$ we mean the largest such count over all inputs of size $n$. Fixing the worst case gives a guarantee that holds for every input, not only for convenient ones.

Counting operations removes the machine, but it still produces an expression like $T(n) = 5n + 3$ whose coefficients depend on the counting convention. Two observations let us discard them. The coefficients shift with the convention and the hardware, so they say nothing about the algorithm itself. And for large $n$ the fastest-growing term dominates the rest, so the constant 3 is negligible against $5n$ and the factor 5 only rescales a quantity that is already linear in $n$. What remains after both observations is the order of growth, the form of $T(n)$ up to a constant factor and for large $n$.

Asymptotic notation makes the phrase "up to a constant factor and for large $n$" precise. We work with functions $f, g \colon \mathbb{N} \to \mathbb{R}_{\geq 0}$, since operation counts are nonnegative.

**Definition 1.1** (Big-O)**.** $f(n) = O(g(n))$ if there exist constants $c > 0$ and $n_0 \in \mathbb{N}$ such that

$$f(n) \leq c\, g(n) \quad \text{for all } n \geq n_0$$

The constant $c$ absorbs the counting convention and the hardware. The threshold $n_0$ discards small inputs, where startup costs and lower-order terms can dominate but where the running time is irrelevant anyway. Read the definition as follows: past input size $n_0$, the function $f$ never exceeds a fixed multiple of $g$. So $g$ bounds the growth of $f$ from above.

**Definition 1.2** (Big-Omega)**.** $f(n) = \Omega(g(n))$ if there exist constants $c > 0$ and $n_0 \in \mathbb{N}$ such that

$$c\, g(n) \leq f(n) \quad \text{for all } n \geq n_0$$

This reverses the inequality of Big-O. Past input size $n_0$, the function $f$ is at least a fixed multiple of $g$, so $g$ bounds the growth of $f$ from below.

**Definition 1.3** (Big-Theta)**.** $f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$. Equivalently, there exist constants $c_1, c_2 > 0$ and $n_0 \in \mathbb{N}$ such that

$$c_1\, g(n) \leq f(n) \leq c_2\, g(n) \quad \text{for all } n \geq n_0$$

A Big-Theta bound sandwiches $f$ between two fixed multiples of $g$. When $f = \Theta(g)$ the two functions grow at the same rate up to constant factors, and $g$ is a tight description of the growth of $f$. This is the bound we look for on an algorithm, since it locates the running time between two known multiples of a single function.

**The three bounds share one pattern**

Each claim is an existential statement about constants. To establish $f = O(g)$ we exhibit one pair $(c, n_0)$ for which the inequality holds. To refute it we show that no such pair can exist. The constants need not be small or tight, only to exist. Big-O gives an upper bound, Big-Omega a lower bound, and Big-Theta both at once.

**Theorem 1.1** (Limit test)**.** *Let $f$ and $g$ be positive for large $n$, and suppose $L = \lim_{n\to\infty} f(n)/g(n)$ exists in $[0, \infty]$. If $0 < L < \infty$ then $f = \Theta(g)$. If $L = 0$ then $f = O(g)$ and $f \neq \Theta(g)$. If $L = \infty$ then $f = \Omega(g)$ and $f \neq \Theta(g)$.*

*Proof.* Suppose $0 < L < \infty$ and set $\varepsilon = L/2$. By the definition of the limit there is an $n_0$ with $|f(n)/g(n) - L| < L/2$ for all $n \geq n_0$. Then $L/2 < f(n)/g(n) < 3L/2$, and multiplying by $g(n) > 0$ gives $(L/2)\, g(n) < f(n) < (3L/2)\, g(n)$. These are the two Big-Theta bounds with $c_1 = L/2$ and $c_2 = 3L/2$. If $L = 0$, then for every $\varepsilon > 0$ we have $f(n) < \varepsilon\, g(n)$ past a threshold, which gives $f = O(g)$ with no matching lower bound. If $L = \infty$, then $f(n)/g(n)$ exceeds any fixed constant past a threshold, which gives $f = \Omega(g)$ with no matching upper bound. □

**Example 1.1.** Let $f(n) = 3n^2 + 5n + 2$ and $g(n) = n^2$. For $n \geq 1$,

$$3n^2 \leq 3n^2 + 5n + 2 \leq 3n^2 + 5n^2 + 2n^2 = 10n^2$$

The lower bound holds because the dropped terms $5n + 2$ are nonnegative. The upper bound replaces each lower-order term by a multiple of $n^2$, valid once $n \geq 1$. So $c_1 = 3$, $c_2 = 10$, and $n_0 = 1$ satisfy the Big-Theta definition, and $f = \Theta(n^2)$. The limit test confirms it faster: $f(n)/g(n) = 3 + 5/n + 2/n^2 \to 3$.

**Example 1.2.** $n = O(n^2)$, since $n \leq n^2$ for $n \geq 1$. The reverse fails: $n \neq \Omega(n^2)$, because $c\, n^2 \leq n$ rearranges to $c\, n \leq 1$, which breaks once $n > 1/c$, for any fixed $c > 0$. Hence $n \neq \Theta(n^2)$. An upper bound alone does not force a matching rate.

*Remark* 1.1. The equality in $f(n) = O(g(n))$ is one-directional. It means that $f$ belongs to the set $O(g)$, so the equals sign reads as "is". We never write $O(g(n)) = f(n)$.

*Remark* 1.2. Big-Omega describes a function, so it applies to whichever running time we name. Stating that an algorithm's worst-case time is $\Omega(n \log n)$ lower-bounds the worst case. It does not describe the best case, and the two are often different.

**Growth hierarchy**

For large $n$, the common running times are ordered by growth,

$$1 \prec \log n \prec \sqrt{n} \prec n \prec n \log n \prec n^2 \prec n^3 \prec 2^n \prec n!$$

where $a \prec b$ means $a = O(b)$ and $a \neq \Theta(b)$. A polynomial-time algorithm runs in $O(n^k)$ for some fixed $k$. An exponential one runs in $2^n$ or worse and leaves the range of direct computation near $n = 40$, where $2^n$ exceeds $10^{12}$.

The brute-force solution to many of these problems is exponential, so the aim throughout is a polynomial-time method with a provable bound.

### The three paradigms

Greedy algorithms build a solution one step at a time. At each step we make the choice that looks best in isolation and never revise it. The work is in proving that this local choice, called a safe move, is consistent with some globally optimal solution. When a safe move exists the algorithm is usually a single pass or a sort followed by a pass.

Divide and conquer splits an instance into smaller instances of the same problem, solves those recursively, and combines their answers. The running time follows from a recurrence that weighs the cost of splitting and combining against the cost of the subproblems. Binary search, merge sort, and the inversion-counting and closest-pair problems all take this form.

Dynamic programming applies when the subproblems overlap, so the same smaller instance is needed many times. We solve each subproblem once, store its answer in a table, and build larger answers from smaller ones. Reusing stored answers turns an exponential recursion into a polynomial table fill.

## 1.1 Fibonacci Sequence Algorithms

The $n$th number in the Fibonacci sequence follows:

$$F(n) = F(n-1) + F(n-2) \qquad F(0) = 0 \quad F(1) = 1$$

We can then consider a few problems using this sequence.

### 1.1.1 Fibonacci Number

Suppose we wanted to write a simple program to calculate $F(n)$. First we consider the recursive approach, which most closely resembles the formal definition.

```
# Recursive
def fib(n):
    if n <= 1:
        return n

    return Fib(n-1) + Fib(n-2)
```

This recursive form runs in $O(\varphi^n)$ time and $O(n)$ stack depth, where $\varphi$ is the golden ratio. Two separate limits make it unusable. The time is exponential because each call recomputes the same subcases, so for $n$ in the low hundreds the computation takes longer than is feasible. The stack depth is linear because the leftmost chain of calls descends $n$ levels before any of them returns, so in Python it raises `RecursionError` once $n$ passes the recursion limit near 1000. Recording each subcase once removes the recomputation. We can do this with a simple `for` loop or with memoization, meaning we store computed values for $O(1)$ lookup:

```python
# For loop
def fib(n):
    prev, current = 0, 1

    for _ in range(n - 1):
        prev, current = current, prev + current

    return current
```

```python
# Memoization
def fib(n):
    memo = {0:0, 1:1}

    def fib_helper(n):
        if n not in memo:
            memo[n] = fib_helper(n-1) + fib_helper(n-2)

        return memo[n]

    return fib_helper(n)
```

Both versions run in $O(n)$ time. The `for` loop keeps two values, so its space is $O(1)$. Memoization keeps all $n$ values, so its space is $O(n)$, and it adds function-call and dictionary overhead. It also recurses $O(n)$ deep, so like the naive version it raises `RecursionError` for large $n$, while the loop does not. We still show it because it generalizes to problems where the dependency between subcases is not a simple sequential chain.

### 1.1.2 Last digit of a Fibonacci number

Again, we can solve this naively, first calling `fib(n)` and then taking the last digit:

```python
def fib_last_digit(n):
    return fib(n) % 10
```

This is correct but slow for large $n$. It builds the full value of $F(n)$, which has on the order of $n/5$ digits, so each addition scales with the digit count rather than being constant.

Since we only want the last digit, we can take the modulus *inside* the loop. The last digit of a sum depends only on the last digits of the addends, since

$$(a + b) \bmod 10 = \big((a \bmod 10) + (b \bmod 10)\big) \bmod 10$$

so reducing at every step gives the same units digit as reducing at the end. Both variables then stay in $\{0, \ldots, 9\}$, so each addition is $O(1)$ and the routine runs in $O(n)$ time and $O(1)$ space:

```
def fib_last_digit(n):
    if n <= 1:
        return n

    previous = 0
    current  = 1

    for _ in range(n - 1):
        previous, current = current, (previous + current) % 10

    return current
```

### 1.1.3 Huge Fibonacci number mod $m$

Next we consider $F(n) \bmod m$ for very large $n$. We have already established that computing a giant Fibonacci number is unwieldy as $n$ grows, even when each step is a single addition. Reducing mod $m$ inside the loop keeps every value below $m$, so each addition is $O(1)$, but the loop still runs $n$ times. For $n$ up to $10^{18}$ even $O(n)$ is unacceptable, so we need a strategy whose cost does not grow with $n$.

We use the Pisano period: the sequence $F(n) \bmod m$ is periodic with period $\pi(m)$. A period must exist by the pigeonhole principle. Each term reduces to a residue, meaning a remainder in $\{0, 1, \ldots, m-1\}$. There are only $m^2$ ordered pairs of consecutive residues, so among the first $m^2+1$ pairs two must coincide.

The cycle returns to the start rather than to some later pair. The recurrence is also invertible mod $m$, since $F(k-1) = F(k+1) - F(k)$, so no pair has two different predecessors. The first repeated pair is therefore the seed pair $(0, 1)$, which makes the sequence periodic from the beginning.

Knowing the sequence begins with 0 and 1, we track it until those two values reappear consecutively. The loop is bounded by $m^2$ steps. Once we have the period, we reduce $n$ modulo it and return the stored value at that index. If $m << n$ then the resulting $O(m^2)$ time will be much better than $O(n)$:

```
def fib_huge_array(n: int, m: int) -> int:
    if n <= 1:
        return n

    sequence = [0, 1]

    for _ in range(m * m):
        next_val = (sequence[-1] + sequence[-2]) % m
        sequence.append(next_val)

        if sequence[-2] == 0 and sequence[-1] == 1:
            period_length = len(sequence) - 2
            return sequence[n % period_length]

    return -1
```

### 1.1.4 Last digit of the sum of Fibonacci numbers

Blah

# 2 Data Structures

# 3 Algorithms on Graphs

# 4 Algorithms on Strings

# 5 Advanced Algorithms and Complexity

# 6 Genome Assembly (Capstone)